

Manipulación del DOM con JavaScript

Introducción

Cuando comenzamos en el mundo del desarrollo web, normalmente empezamos aprendiendo HTML y CSS. Con esto podemos crear páginas **estáticas**: páginas que no pueden realizar tareas dinámicas ni personalizadas dependiendo de acciones del usuario.

Si añadimos **JavaScript**, podremos crear páginas **dinámicas**, aprovechando la potencia de un lenguaje de programación para crear experiencias más completas.

¿Qué es el DOM?

DOM significa **Document Object Model**. Es la estructura de un documento HTML, formada por múltiples etiquetas anidadas formando un **árbol** de etiquetas relacionadas entre sí.

En JavaScript, el DOM es esa estructura de árbol mediante la cual podemos acceder y modificar los elementos del HTML: añadir nuevas etiquetas, modificar o eliminar otras, cambiar atributos, añadir clases, modificar contenido de texto, etc.

El objeto document

Desde el navegador, accedemos al DOM a través del objeto `document`, que representa el árbol DOM de la página actual.

Búsqueda de elementos en el DOM

Métodos tradicionales

Método	Descripción	Devuelve
<code>getElementById(id)</code>	Busca un elemento por su <code>id</code>	Un elemento o <code>null</code>
<code>getElementsByClassName(class)</code>	Busca elementos por su clase	<code>HTMLCollection</code>
<code>getElementsByName(name)</code>	Busca elementos por atributo <code>name</code>	<code>NodeList</code>
<code>getElementsByTagName(tag)</code>	Busca elementos por etiqueta	<code>HTMLCollection</code>

Métodos modernos (recomendados)

Método	Descripción	Devuelve
<code>querySelector(selector)</code>	Primer elemento que coincide con el selector CSS	Un elemento o <code>null</code>
<code>querySelectorAll(selector)</code>	Todos los elementos que coinciden con el selector CSS	<code>NodeList</code> (estática)

Ventaja: Puedes usar selectores CSS completos, como `".main .info"`, `"div > p:first-child"`, etc.

javascript

```
const page = document.querySelector("#page"); // por id
const info = document.querySelector(".main .info"); // por clase anidada
const allDivs = document.querySelectorAll("div"); // todos los divs
```

HTMLCollection vs Array

Los métodos tradicionales devuelven `HTMLCollection`, que es una **colección viva**: si modificas el DOM, la colección se actualiza automáticamente.

javascript

```
// HTMLCollection (viva)
const collection = document.getElementsByTagName("div");
collection.length; // 63
collection[62].remove(); // eliminamos un div del DOM
collection.length; // 62 (se actualiza automáticamente)
```

```
// Convertir a Array (estático)
const array = [...document.getElementsByTagName("div")];
array.length; // 63
array[62].remove();
array.length; // 63 (no se actualiza)
```

Para convertir a un array real y poder usar `forEach`, `map`, `filter`:

```
javascript

const elementos = Array.from(document.getElementsByClassName("clase"));
// o también:
const elementos2 = [...document.getElementsByTagName("p")];
```

Propiedades de texto

`.textContent`

Devuelve o modifica **solo el texto** de un elemento, ignorando etiquetas HTML. Es la propiedad recomendada.

```
javascript

const element = document.querySelector(".message");
element.textContent = "Hola a todos";
console.log(element.textContent); // "Hola a todos"
```

`.innerText`

Similar a `.textContent`, pero solo devuelve el texto **visualmente renderizado** (respeta CSS, elementos ocultos, etc.).

```
javascript

const element = document.querySelector(".container");
console.log(element.innerText); // solo lo visible en pantalla
```

.outerText

Igual que `.innerText` para leer, pero al escribir **reemplaza el elemento completo**, no solo su contenido.

javascript

```
// <div class="container">texto</div>
element.outerText = "Nuevo texto";
// Resultado: "Nuevo texto" (el <div> desaparece)
```

Propiedades HTML

.innerHTML

Devuelve o modifica el **contenido HTML interno** de un elemento. **El HTML se interpreta.**

javascript

```
const element = document.querySelector(".message");
element.innerHTML = "<strong>Texto en negrita</strong>";
// Se renderiza como negrita
```

.outerHTML

Devuelve o modifica el elemento **incluyéndose a sí mismo**.

javascript

```
// <div class="data">contenido</div>
console.log(element.outerHTML); // toda la etiqueta <div>
element.outerHTML = "<h1>Nuevo título</h1>";
// El <div> es reemplazado por el <h1>
```

Advertencia de seguridad

Nunca insertes contenido de usuario sin sanitizar usando `innerHTML`, ya que puede haber **inyección de código malicioso** (XSS).

javascript

// PELIGROSO si el texto viene del usuario:

```
element.innerHTML = usuarioInput;
```

Crear y eliminar elementos

createElement(tag) – Crear un nuevo elemento

javascript

```
const nuevoDiv = document.createElement("div");
nuevoDiv.textContent = "Soy un div nuevo";
nuevoDiv.classList.add("mi-clase");
```

appendChild(element) – Insertar como último hijo

javascript

```
const contenedor = document.querySelector(".contenedor");
contenedor.appendChild(nuevoDiv);
```

insertAdjacentHTML(posicion, html) – Insertar HTML en una posición específica

javascript

```
contenedor.insertAdjacentHTML("beforeend", "<span>final</span>");
```

Posiciones válidas: "beforebegin", "afterbegin", "beforeend", "afterend".

remove() – Eliminar un elemento

javascript

```
const elemento = document.querySelector(".borrar");
elemento.remove();
```

removeChild(element) – Eliminar un hijo específico

javascript

```
const padre = document.querySelector(".lista");
const hijo = padre.querySelector("li:first-child");
```

```
padre.removeChild(hijo);
```

Atributos HTML

Acceso directo (más sencillo)

```
javascript  
  
const element = document.querySelector("div");  
element.id = "miId";  
element.className = "clase1 clase2"; // ¡class, no class!  
element.style.color = "red";
```

Métodos para atributos (más potente)

```
javascript  
  
element.getAttribute("data-info");  
element.setAttribute("data-info", "valor");  
element.hasAttribute("data-info");  
element.removeAttribute("data-info");
```

classList – Manejo avanzado de clases

En lugar de modificar directamente `className`, usa `classList`:

```
javascript  
  
element.classList.add("nueva-clase");  
element.classList.remove("vieja-clase");  
element.classList.toggle("activo"); // si existe la quita, si no la añade  
element.classList.contains("activo"); // true / false  
element.classList.replace("vieja", "nueva");
```

Eventos en JavaScript

Los eventos permiten responder a las acciones del usuario.

Formas de asignar eventos

Opción 1: Atributo HTML (no recomendada)

html

```
<button onclick="alert('click')">Click</button>
```

Opción 2: Propiedad del elemento (limitada)

javascript

```
boton.onclick = function() { alert("click"); };
```

Opción 3: `addEventListener` (recomendada)

javascript

```
const boton = document.querySelector("button");
boton.addEventListener("click", () => {
  alert("¡Click!");
});
```

Ventajas de `addEventListener`:

- Permite múltiples funciones para el mismo evento.
- Mayor control (opciones como `once`, `capture`).
- Mejor organización del código.

Tipos de eventos más comunes

Eventos de ratón

Evento	Descripción
click	Clic en un elemento
dblclick	Doble clic
mouseenter	Cursor entra al elemento
mouseleave	Cursor sale del elemento
mousedown	Botón del mouse presionado
mouseup	Botón del mouse liberado

Evento Descripción

mousemove	Mouse se mueve sobre el elemento
-----------	----------------------------------

Eventos de teclado

Evento Descripción

keydown	Tecla presionada
---------	------------------

keyup	Tecla liberada
-------	----------------

 `keypress` está obsoleto, no lo uses.

Eventos de formulario

Evento Descripción

submit	Formulario enviado
--------	--------------------

change	Valor de un input cambia
--------	--------------------------

input	Se escribe en un campo (en tiempo real)
-------	---

focus	Elemento recibe foco
-------	----------------------

blur	Elemento pierde foco
------	----------------------

Eventos de ventana

Evento Descripción

load	Página completamente cargada
------	------------------------------

resize	Ventana cambia de tamaño
--------	--------------------------

scroll	Scroll de la ventana
--------	----------------------

Ejemplo completo

html

```
<button id="miBoton">Haz clic</button>
```

```
<p id="mensaje"></p>
```

```
<script>
```

```
const boton = document.getElementById("miBoton");
```

```
const mensaje = document.getElementById("mensaje");
```

```
boton.addEventListener("click", () => {  
  mensaje.textContent = "¡Botón clickeado!";  
});  
</script>
```

Resumen de buenas prácticas

Práctica	Recomendación
Buscar elementos	Usa <code>querySelector</code> / <code>querySelectorAll</code>
Modificar texto	Usa <code>.textContent</code>
Modificar HTML	Usa <code>.innerHTML</code> con precaución
Modificar clases	Usa <code>.classList</code>
Asignar eventos	Usa <code>addEventListener</code>
Insertar elementos	Usa <code>createElement</code> + <code>appendChild</code> o <code>insertAdjacentHTML</code>
Seguridad	Nunca inserts texto de usuario directamente en <code>innerHTML</code>

Glosario rápido

Término	Significado
DOM	Estructura de árbol del documento HTML
<code>document</code>	Objeto que representa la página actual
<code>HTMLCollection</code>	Colección viva de elementos (se actualiza sola)
<code>NodeList</code>	Colección generalmente estática
<code>querySelector</code>	Busca con selectores CSS
<code>classList</code>	Manejo avanzado de clases CSS